MAJOR PROJECT · IMPLEMENTATION PLAN

Building Sentra: A Phase-wise Plan from Empty Folder to Viva

Eleven phases, each with concrete tasks, code to write, and an exit test you can actually run.

Document 2 of 2 — Implementation Plan

Companion document — Concepts, Architecture & Viva Preparation

Stack — Java 21 · Spring Boot 4.1 · PostgreSQL 16 · React + Vite

Prepared for — Kshitij, Swami Keshvanand Institute of Technology

Date — 22 September 2026

Read Document 1 first — this plan assumes its vocabulary

Contents

1. Before You Write Any Code

- 1.1 The assumptions this plan makes — change these first if they are wrong
- 1.2 Stack decisions, with versions and reasons
- 1.3 What to install
- 1.4 Repository structure
- 1.5 Six rules that decide whether this project finishes

2. The Phases

- Phase 0 — Skeleton and deployment pipeline
- Phase 1 — One check, end to end, no scheduler
- Phase 2 — The engine: scheduler, probes, runs
- Phase 3 — Incidents, state machine, hysteresis
- Phase 4 — Alerting
- Phase 5 — Authentication, multi-tenancy, SSRF guard
- Phase 6 — Heartbeat monitoring (the first push feature)
- Phase 7 — TLS, domain and TCP checks (the cheap three)
- Phase 8 — Multi-step synthetic transactions
- Phase 9 — Deploy correlation and the public status page
- Phase 10 — Retention, hardening, documentation, demo

3. Reference

- 3.1 Complete API surface
- 3.2 Complete database schema
- 3.3 Dependencies (pom.xml)
- 3.4 Testing strategy
- 3.5 Frontend screens

4. Delivery

- 4.1 Week-by-week calendar
- 4.2 If you only have eight weeks — the cut list
- 4.3 Risk register
- 4.4 What goes in the report
- 4.5 The eight-minute demo script

1. Before You Write Any Code

Twenty minutes of decisions here saves two weeks later.

1.1 The assumptions this plan makes

You did not tell me your timeline or whether you have teammates, so I have assumed the most common case and built the plan around it. **If either assumption is wrong, adjust before starting** — §4.2 tells you exactly what to cut for a shorter timeline, and this section tells you how to split the work for a team.

Assumption	What I assumed	If it is different
Timeline	12 weeks, i.e. late September to mid-December 2026	8 weeks or fewer → follow the cut list in §4.2. More than 14 → add the on-call escalation module from Document 1 §6.5.
Team size	Solo, roughly 10–12 hours a week	See the split below.
Your level	Comfortable with Java and basic Spring Boot; new to schedulers, security and concurrency	If you have built Spring Boot APIs before, Phases 0–1 collapse into three days.
Deliverables	Working system + report + viva demo	If a research paper is also required, the SSRF and scheduler sections are your strongest material.

> IF YOU HAVE TEAMMATES
>
> Do **not** split by feature — two people building two check types in parallel before the engine exists will produce two incompatible engines. Split by layer, after Phase 3 is done and merged:
>
> - **Person A (you)** — engine, probes, incidents, security. The hard half, and the half the viva is about.
> - **Person B** — React dashboard, charts, status page, plus the API client layer.
> - **Person C, if there is one** — notifications, retention job, deployment, tests, documentation.

1.2 Stack decisions, with versions and reasons

Choice	Version	Why, and what to say if asked
Java	21 (LTS)	Long-term support, and it has virtual threads — which matter here because your probe workload is almost pure blocking I/O. Spring Boot 4 requires Java 17 as a minimum and supports well beyond 21, but 21 is the safe LTS choice with full tooling support.
Spring Boot	4.1.x	The current supported line (4.1 released June 2026, built on Spring Framework 7). Note that **Spring Boot 3.5 reached open-source end of life on 30 June 2026** — so if you follow an older tutorial, expect version drift. See the warning below.

Choice	Version	Why, and what to say if asked
PostgreSQL	16 or newer	You need three things specifically: a real `JSONB` column type for check config, efficient time-range indexing for the run table, and `SELECT ... FOR UPDATE SKIP LOCKED` if you ever discuss scaling. MySQL would work but you would be arguing for it rather than from it.
Flyway	current	Version-controlled schema. From Flyway 10 onward the PostgreSQL support ships as a separate artifact, so add `flyway-database-postgresql` alongside `flyway-core` or startup will fail with a confusing error.
React	with Vite	Vite for fast dev builds. Use plain `fetch` or axios plus TanStack Query — polling and caching are exactly what your dashboard needs and it removes a lot of state-management code.
Charts	Recharts	Enough for uptime bars and latency lines without a week of D3.
Build	Maven	Either is fine; Maven has more Spring documentation. Pick one and never discuss it again.
Tests	JUnit 5 + Testcontainers	Testcontainers runs a real PostgreSQL in Docker for integration tests, so you test against the database you actually deploy on rather than H2.

> DECIDE THIS CONSCIOUSLY ON DAY ONE
>
> **Spring Boot 4.1 versus 3.5.** 4.1 is current and supported; 3.5 is end-of-life but has several years of tutorials, StackOverflow answers and blog posts written against it. If you hit a wall on Boot 4 and cannot find an answer, that is a real cost to a student project on a deadline.
>
> My recommendation is **4.1**, because "I used the current supported release and noted that 3.5 went end-of-life in June" is a good line in a viva, and because the annotations you will use most (`@RestController` , `@Scheduled` , JPA, Spring Security's filter-chain configuration) are stable across both. Whichever you pick, **pick it before Phase 0 and never change it mid-project.**

1.3 What to install

- **JDK 21** — Temurin or any distribution; verify with `java -version` .
- **Docker Desktop** — for PostgreSQL locally and for Testcontainers. Do not install PostgreSQL natively; a container is disposable.
- **IntelliJ IDEA Community** — free, and its Spring support is worth it.
- **Node.js LTS** — for the React frontend.
- **A REST client** — Bruno, Insomnia or IntelliJ's built-in `.http` files. You will be calling your own API constantly before any UI exists.

```
# Your local database, in one command. Run this once and forget it.
docker run -d --name sentra-db \
  -e POSTGRES_PASSWORD=devpass -e POSTGRES_DB=sentra \
  -p 5432:5432 postgres:16
```

1.4 Repository structure

One repository, two folders. A monorepo is simpler to submit, simpler to deploy as a single artifact, and gives you one commit history to show.

```
sentra/
├─ backend/
│  ├─ src/main/java/in/sentra/
│  │  ├─ SentraApplication.java
│  │  ├─ check/          // Check entity, repo, service, controller, CheckType
│  │  ├─ probe/          // Probe interface + one class per check type
│  │  ├─ engine/         // Scheduler, CheckRunner, EvaluationService
│  │  ├─ incident/       // Incident entity + state machine
│  │  ├─ alert/          // AlertRule, Notifier strategy, delivery log
│  │  ├─ ingest/         // /ping and /webhook controllers (push endpoints)
│  │  ├─ status/         // public status page controller
│  │  ├─ security/       // JWT filter, SecurityConfig, SsrfGuard
│  │  └─ common/         // exceptions, Clock bean, error handler
│  ├─ src/main/resources/
│  │  ├─ application.yml
│  │  └─ db/migration/   // V1__init.sql, V2__incidents.sql, ...
│  └─ src/test/java/...
├─ frontend/           // React + Vite
├─ docs/               // report, diagrams, screenshots, this plan
├─ docker-compose.yml
└─ README.md
```

Package **by feature, not by layer**. A `check/` package containing the entity, repository, service and controller is easier to navigate than four packages called `entity/`, `repository/`, `service/`, `controller/` each holding fragments of nine unrelated features. It also makes it obvious when a feature is reaching into another's internals.

1.5 Six rules that decide whether this project finishes

#	Rule	Why
1	**Build one check type all the way through before adding a second**	The engine is the hard part. Once HTTP checks work end to end — scheduled, evaluated, alerting — the other four pull types cost a day each. Building five probes before the engine exists means five probes with nothing to run them.
2	**Deploy in week one, even when it does nothing**	Deployment problems discovered in week 11 are catastrophic; discovered in week 1 they are an afternoon. It also means the platform starts accumulating real uptime history early, which becomes your best demo evidence.
3	**Commit every day, with messages a human can read**	Your commit history is submittable evidence that this was built steadily rather than copied. It is also the only thing that saves you when something that worked yesterday does not today.
4	**Write the test that proves the tricky thing**	You do not need 80% coverage. You need about ten tests that prove the hard parts: hysteresis thresholds, tenant isolation, idempotent pings, SSRF rejection. Those ten are worth more in a viva than a hundred trivial ones.

#	Rule	Why
5	**Keep a decisions log**	One markdown file. Every time you choose between two options, write three lines: what you chose, what you rejected, why. At report-writing time this file *is* your design chapter, and in the viva it is the difference between recalling and reconstructing.
6	**The UI comes after the API works**	Test every endpoint with a REST client first. Debugging a backend through a half-built React form is the slowest possible way to work, and it is how most student projects lose their final fortnight.

2. The Phases

Each phase has a goal, a task list, and an exit test. Do not start the next phase until the current one's exit test passes.

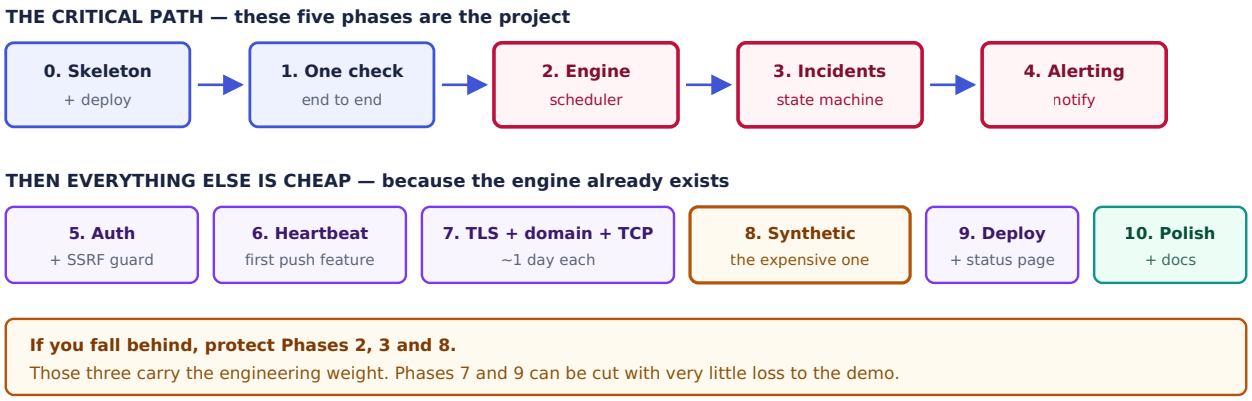


Figure 2.1 — Phase dependency and where the real effort sits.

PHASE 0

Skeleton and deployment pipeline

Week 1 · ~8 hours · Produces: an empty app running in production

> **Goal:** a Spring Boot application that starts, connects to PostgreSQL, answers one endpoint, and is already deployed to a public URL — before it does anything useful.

Why this comes first

Deployment is the single most common source of week-11 disasters. Doing it while the application has zero complexity means any problem you hit is definitely an infrastructure problem, not your code. From this point on, every subsequent phase ends with something visible on the internet.

Tasks

1. Generate the project at `start.spring.io`: Java 21, Maven, Spring Boot 4.1.x. Dependencies: Web, JPA, PostgreSQL Driver, Validation, Actuator, Flyway.
2. Start PostgreSQL in Docker (§1.3) and configure `application.yml`. Set `spring.jpa.hibernate.ddl-auto: validate` — Flyway owns the schema, Hibernate only checks it matches.
3. Write `V1__init.sql` with two tables for now: `app_user` and `project`.
4. Add a trivial `GET /api/v1/ping` returning `{"status":"ok"}` so you have something to deploy and something to monitor later.
5. Initialise git; write a real README with the project description and how to run it.
6. Deploy. Push to GitHub, connect to your hosting provider, set the database URL as an environment variable, deploy. **Read Document 1 §10.2 before choosing a host** — free tiers that sleep will break your scheduler later, and free databases that expire after 30 days will delete your history mid-project.
7. Create a `docs/decisions.md` and write your first entry: why Spring Boot 4.1 over 3.5, why PostgreSQL.

What you learn

Spring Boot configuration, Flyway migration ordering, environment-variable-driven config (never commit a password), and the deployment loop you will use twenty more times.

EXIT TEST — ALL MUST PASS BEFORE PHASE 1

- ☐ `curl https://<your-app>/api/v1/ping` returns 200 from the public internet, not just localhost
- ☐ `/actuator/health` reports the database as `UP`
- ☐ Flyway created `flyway_schema_history` and both tables exist in the deployed database
- ☐ Nothing secret is committed — grep your repository for the word `password`

PHASE 1

One check, end to end, with no scheduler

Week 2 · ~10 hours · Produces: create a check and run it manually

> **Goal:** you can create an HTTP check through the API and trigger it by hand, and the result is stored. No scheduling, no incidents, no UI.

Why this shape

This is a **vertical slice** — one thin path through every layer — rather than a horizontal one where you build all the entities, then all the repositories, then all the services. A vertical slice means you have something that works at the end of week 2, and every layer has been forced to meet the others early.

Tasks

1. `V2__checks.sql` : the `checks` and `check_runs` tables (full DDL in §3.2).
2. `Check` entity with the `CheckType` enum and the `JSONB` config column. Only `HTTP` exists so far.
3. `CheckRepository` , `CheckService` , `CheckController` with create, list, get, delete.
4. Request and response DTOs with Bean Validation. **Never expose the entity directly** — it leaks your schema and creates accidental API changes when you refactor.
5. Define the `Probe` interface and write `HttpProbe` (Document 1 §5.2), with explicit connect and read timeouts.
6. Add `POST /api/v1/checks/{id}/run` — a manual trigger. This exists for your own testing now, and becomes a genuine “run now” UI button later.
7. A `@ControllerAdvice` global exception handler returning consistent JSON errors. Do this now, once, rather than retrofitting it across twelve controllers in week 10.

```
// The extension point. Write it now, even with only one implementation,
// because designing it against one type produces a bad abstraction.
public interface Probe {
    CheckType supports();
    ProbeResult execute(Check check);
}
```

What you learn

JPA entity mapping including JSON columns, DTO separation, Bean Validation, centralised error handling, and making HTTP calls from a Spring service with timeouts.

EXIT TEST

- ☐ `POST /api/v1/checks` with a URL returns 201 and a body containing the new id
- ☐ `POST /api/v1/checks/{id}/run` against a real site stores a `check_runs` row with a latency value
- ☐ Pointing a check at a deliberately broken URL stores a failure with a *categorised* reason, not a stack trace
- ☐ Posting a check with no URL returns 400 with a readable message, not a 500

PHASE 2

The engine — scheduler, dispatch, runs

Week 3 · ~10 hours · Produces: the platform starts working by itself

> **Goal:** checks run automatically on their own intervals, with nobody touching anything. This is the phase where your project stops being CRUD.

Tasks

1. Add `next_run_at`, `interval_seconds` and `enabled` to the check table, plus the index that makes the due query fast:

   ```
   CREATE INDEX idx_checks_due ON checks (next_run_at) WHERE enabled;
   ```

   (A partial index — it only contains enabled rows, which is all the scheduler ever asks about.)

2. Enable scheduling with `@EnableScheduling` and write `CheckScheduler` exactly as in Document 1 §6.1 — **claim before probing**.
3. Configure a bounded `ThreadPoolTaskExecutor` for probes, or enable virtual threads. Never probe on the scheduler thread.
4. Build the probe registry: inject `List<Probe>` and index it into a `Map<CheckType, Probe>` at startup.

   ```
   ProbeRegistry(List<Probe> probes) {
       this.byType = probes.stream()
           .collect(toMap(Probe::supports, identity()));
   }
   ```

5. Add jitter when computing the next run, so checks do not bunch at the top of the minute.
6. Inject a `Clock` bean instead of calling `Instant.now()` directly — this is what makes Phase 3 testable.
7. Log every dispatch batch at DEBUG and every failure at WARN. Background code with no logging is invisible when it breaks.

> **THE MISTAKE ALMOST EVERYONE MAKES HERE**
>
> Updating `next_run_at` *after* the probe returns instead of before dispatch. A probe that takes 30 seconds on a 10-second tick then gets launched three more times concurrently, and you will see mysterious duplicate rows. Claim first, probe second.

What you learn

Spring's scheduling abstraction, thread pools and why pool size is not tied to CPU count for I/O-bound work, partial indexes, and dependency injection of a collection of implementations — which is the mechanism that makes your design extensible.

EXIT TEST

- ☐ Create a 60-second check, walk away for ten minutes, come back to ten run rows — with no interaction
- ☐ Disabling a check stops it running within one tick
- ☐ A check pointed at a URL that hangs does not delay any other check — verify with a deliberately slow endpoint
- ☐ Restarting the application resumes all schedules correctly, because they live in the database
- ☐ Deployed and running continuously — from here on it accumulates real history

PHASE 3

Incidents, the state machine and hysteresis

Week 4 · ~10 hours · Produces: failures become meaningful events

> **Goal:** a run of failures opens exactly one incident; a run of successes closes it. This is the intellectual core of the platform.

Tasks

1. `V4__incidents.sql`: the `incidents` table, plus `state`, `consecutive_failures` and `consecutive_successes` columns on `checks`.
2. `CheckState` enum: `PENDING`, `HEALTHY`, `DEGRADED`, `DOWN`, `PAUSED`.
3. `EvaluationService` — given a check and a probe result, update the counters, decide the new state, and open or close an incident. Keep this class pure of HTTP and of notification concerns; it should be callable from a unit test with no Spring context at all.
4. Make the thresholds configurable per check (`failure_threshold`, `success_threshold`), defaulting to 3 and 2.
5. Guard the transitions explicitly. Write the state machine as a method that *rejects* illegal transitions rather than a chain of `if` statements that silently permits them.
6. Add `GET /api/v1/incidents` and `GET /api/v1/checks/{id}/incidents`.
7. **Write the tests now**, while the logic is fresh. This is the highest-value test file in the project:
   - two failures then a success → no incident ever opened
   - three consecutive failures → exactly one incident, state `DOWN`
   - five more failures → still exactly one incident, not five
   - one success while down → incident stays open (M = 2 not yet met)
   - two successes → incident closes with a `closed_at` timestamp

What you learn

State machines, hysteresis, transactional boundaries (opening an incident and updating the check must be atomic), and how to unit-test time-dependent logic by injecting a fixed `Clock`.

EXIT TEST

- ☐ Point a check at a URL, break it, and watch exactly one incident appear after the third failure
- ☐ Fix the URL and watch the incident close with a duration, after the second success
- ☐ All five unit tests above pass
- ☐ The incident list endpoint returns correct open and closed incidents with durations

PHASE 4

Alerting

Week 5 · ~9 hours · Produces: the platform tells a human

> **Goal:** when an incident opens, a notification is sent exactly once; when it closes, a recovery notice is sent. A long outage produces reminders with widening gaps, not a flood.

Tasks

1. `V5__alerts.sql` : `alert_rules` and `alert_deliveries`, with the unique constraint that makes double-sending impossible:

   ```
   CREATE UNIQUE INDEX uq_alert_once
       ON alert_deliveries (incident_id, channel, transition_type);
   ```

2. Define the strategy interface and write two implementations:

   ```
   public interface Notifier {
       Channel channel();                      // EMAIL, WEBHOOK
       void send(AlertMessage message, String target);
   }
   ```

   Start with `WebhookNotifier` — it is a single HTTP POST and you can test it against a free request-inspection endpoint in minutes, with no account signup and no deliverability problems. Add email after it works.

3. **Use an email provider's HTTP API, not SMTP.** Several hosts block outbound SMTP ports on free plans (Document 1 §10.3), and you would discover this at the worst possible time. An HTTPS call is never blocked and gives you a delivery ID to record.

4. Hook the notifier into `EvaluationService` via a Spring application event rather than a direct call. The evaluation logic publishes `IncidentOpenedEvent`; a listener sends the alert. This keeps the core logic testable without stubbing a mail client, and it makes adding channels genuinely zero-touch.

5. Reminder job: a scheduled sweep over open incidents, sending a reminder when the backoff interval has elapsed (15 min, 30, 60, then hourly).

6. Record every delivery attempt with its outcome, so a failed send is visible rather than silent.

> **WHY EVENTS RATHER THAN A DIRECT METHOD CALL**
>
> If `EvaluationService` calls the notifier directly, every unit test of your incident logic now needs a mock mailer, and the two concerns become permanently entangled. Publishing an event keeps the decision ("this became DOWN") separate from the consequence ("tell someone"). It is a small change that reads as deliberate architecture in a viva.

What you learn

The strategy pattern applied for real, Spring's application event mechanism, exponential backoff, and using database constraints as correctness guarantees rather than trusting application logic.

EXIT TEST

- ☐ Break a monitored URL → exactly one alert arrives after the third failure, not three
- ☐ Leave it broken for an hour → reminders arrive at widening intervals, not every minute
- ☐ Fix it → one recovery notice arrives
- ☐ Force a duplicate send in code → the unique constraint blocks it and nothing is sent twice
- ☐ An `alert_deliveries` row exists for every message, with its status

PHASE 5

Authentication, multi-tenancy and the SSRF guard

Week 6 · ~11 hours · Produces: a system safe to show other people

> **Goal:** users register and log in, they see only their own data, and nobody can use your server as a proxy into your own network.

> THIS PHASE CONTAINS YOUR BEST VIVA MATERIAL
>
> The SSRF guard is the single most impressive thing in the project (Document 1 §7.3, question Q28). Do not skip it, do not rush it, and write the test that proves it works — that test is a slide in your presentation.

Tasks

1. `V6__auth.sql`: passwords on `app_user`, plus a `project` foreign key on `checks` if it is not already there.
2. Register and login endpoints. Hash passwords with BCrypt — `PasswordEncoder` is provided; never store or log a raw password.
3. JWT issuing and a validation filter. Either `spring-boot-starter-oauth2-resource-server` (more idiomatic, less code) or the jjwt library (more explicit, easier to explain line by line). For a viva, being able to explain it matters, so pick the one you can narrate.
4. Two separate `SecurityFilterChain` beans, ordered by path (Document 1 §7.1): one for `/api/**` requiring JWT, one for `/ping/**`, `/webhook/**` and `/status/**` which are public by design.
5. **Retrofit tenant scoping across every existing endpoint.** Change each repository method to include the owner, e.g. `findByIdForUser(id, userId)`. This is tedious and it is the entire point of the phase — a single endpoint you forget is a working IDOR vulnerability.
6. Write `SsrfGuard` (Document 1 §7.3) and call it in two places: when a check is created, **and again inside the probe immediately before connecting**.
7. Rate-limit the public endpoints. A simple in-memory token bucket per token is sufficient and easy to explain.
8. Tests — these two are non-negotiable:
   - user B requesting user A's check receives **404**, not 403 and certainly not 200
   - creating a check targeting `http://localhost`, `http://169.254.169.254` and `http://10.0.0.1` is rejected in all three cases

What you learn

Spring Security's filter chain, stateless authentication, the difference between authentication and authorisation in code rather than in theory, and a real, named, exploitable vulnerability class and its mitigation.

EXIT TEST

- ☐ Registration, login and an authenticated request all work end to end
- ☐ A request with no token gets 401; with a valid token but someone else's resource, 404
- ☐ Both security tests above pass
- ☐ The ping and status endpoints still work without any token — verify you did not over-secure them

PHASE 6

Heartbeat monitoring — the first push feature

Week 7 · ~8 hours · Produces: cron job monitoring

> **Goal:** a user pastes one line of curl into a cron job, and your platform notices when that job stops running.

Why it is its own phase despite looking simple

Heartbeat is the first feature that inverts the direction of traffic, and the first that detects an *absence* rather than a result. The listener is twenty lines; the sweep, the deadline arithmetic and the idempotency are where the work is.

Tasks

1. Add `HEARTBEAT` to `CheckType`, plus `token`, `grace_period_seconds` and `next_expected_at` columns. Unique index on `token`.
2. Generate tokens with `SecureRandom` or `UUID.randomUUID()`. Never sequential, never derived from the name.
3. `PingController` with `POST /ping/{token}` returning 204, plus the `/start` and `/fail` variants (Document 1 §5.1).
4. `recordPing`: write a run row, push `next_expected_at` to `now + interval + grace`, close any open incident.
5. The miss-detection sweep — a second `@Scheduled` method, separate from the probe scheduler, because it asks a different question.
6. Idempotency: accept an optional `X-Ping-Id` header, store it, and treat a duplicate as success rather than as an error.
7. Handle the `PENDING` state properly, so a newly created heartbeat check does not immediately alert for never having pinged.
8. In the dashboard (or in the API response), return the ready-to-paste integration snippet with the token already substituted. Small touch, big usability difference.
9. Mask the token in access logs.

What you learn

Push-based ingestion, designing for retry and duplication, deadline arithmetic with grace periods, and unauthenticated endpoint hardening.

EXIT TEST

- ☐ A real cron job on your own machine pings successfully and the dashboard shows it
- ☐ Stop the cron job → an incident opens after the interval plus grace elapses, and not before
- ☐ Sending the same ping ID twice creates one run row, not two, and both requests return 204
- ☐ A newly created heartbeat check sits in `PENDING` and does not alert
- ☐ An invalid token returns 404 quickly and is rate-limited under repeated attempts

PHASE 7

TLS, domain and TCP checks — the cheap three

Week 8 · ~9 hours total · Produces: proof that the abstraction holds

> **Goal:** add three entire check types in one week, changing nothing outside the `probe/` package. This is your architectural claim, demonstrated.

> FRAME THIS PHASE DELIBERATELY IN YOUR REPORT
>
> Do not present these as "three more features." Present them as **the experiment that validates the design**: "adding three check types required three new classes and two new enum values; the scheduler, incident engine, alerting and dashboard were not modified." Count the lines of code you had to change outside `probe/` and put the number in your report. If the answer is close to zero, you have evidence rather than an assertion.

TCP (half a day — do this one first)

1. Add `TCP` to the enum; config is host and port.
2. `TcpProbe` using `Socket.connect` with a timeout (Document 1 §5.5).
3. Distinguish refused from timed out in the stored failure reason — they mean different things.
4. Test against a port you know is closed, and one you know is open.

TLS certificate (one to two days — the fiddly one)

1. `TlsProbe` using `SSLSocket`, with **SNI set explicitly** or shared hosts return the wrong certificate.
2. Use a permissive trust manager scoped to this probe only, so an already-expired certificate can still be read (Document 1 §5.3). Comment the code explaining why, because it looks alarming out of context.
3. Store `days_remaining`, issuer and exact expiry.
4. Tiered thresholds (30 / 14 / 7 / 3 / 1 days), alerting on crossings only, not daily.
5. Default interval: once a day. Enforce a sensible minimum — nobody needs a per-minute certificate check.

Domain expiry (one day)

1. `DomainProbe` querying RDAP over HTTPS and reading the `expiration` event.
2. **Test with a `.in` domain early.** RDAP coverage varies among country-code TLDs; if it is unavailable you need the WHOIS fallback, and you would rather know in week 8 than week 11.
3. A distinct `PARSE_FAILURE` outcome — "could not determine expiry" must never be reported as "expiring soon".
4. Once-daily interval, enforced. Registries rate-limit aggressive queriers.

What you learn

Java's TLS APIs and certificate model, raw socket programming, a legacy text protocol alongside its modern JSON replacement, and — most importantly — what a good abstraction feels like when you extend it.

EXIT TEST

- ☐ All three types create, schedule and run through the existing engine with no engine changes
- ☐ A TLS check against your own domain reports a plausible days-remaining figure you can verify in a browser
- ☐ A TLS check against a deliberately expired-certificate test site reports the expiry rather than crashing
- ☐ A domain check returns a real expiry date for a `.com` and for your `.in` domain (or documents the gap)
- ☐ You have counted and recorded the lines changed outside `probe/`

PHASE 8

Multi-step synthetic transactions

Weeks 9–10 · ~18 hours · Produces: the feature that makes the project distinctive

Goal: a user defines a sequence of HTTP steps with assertions, values flow from one step to the next, and the platform reports *which step* failed.

BUDGET TWO FULL WEEKS AND PROTECT THEM

Six of your seven features are variations on one theme. This one is a small interpreter: a data model for steps, a template substitution mechanism, an assertion evaluator, an extraction mechanism, and a UI for building it all. It is where genuine engineering effort goes, and it is the feature an examiner will find most interesting. If you are behind schedule, cut Phase 7 or Phase 9 — not this.

Build it in this order

Step 1 — the data model (2 hours)

Define the step JSON structure (Document 1 §5.6) as Java records with Bean Validation, stored in the existing `config` column. No schema migration is needed at all — which is itself worth pointing out in your report as a payoff from the Phase 1 design decision.

Step 2 — sequential execution, no variables yet (3 hours)

Walk the steps, send each request, stop on the first failure. Record which step index failed. Get this working with a two-step flow against a public test API before adding anything clever.

Step 3 — assertions (3 hours)

Implement four assertion types, each as a small class behind one interface:

Type	Example	Notes
`status`	`{"type":"status","equals":200}`	Support ranges too (`2xx`)
`jsonPath`	`{"path":"$.token","exists":true}`	Use the Jayway JsonPath library rather than writing a parser
`contains`	`{"type":"contains","text":"Order placed"}`	Plain substring on the body
`maxMs`	`{"type":"maxMs","value":2000}`	Catches degradation before it becomes an outage

Step 4 — variable extraction and substitution (4 hours)

The context map and `{{placeholder}}` rendering. This is what turns three requests into a transaction. Two details that will bite you:

- Substitute in the URL, headers **and** body — not just the body.

- A placeholder referencing a variable that was never extracted must fail the step with a clear message, not send the literal string `{{token}}` and produce a baffling 401.

Step 5 — secrets (3 hours)

Test credentials are stored in the database. Encrypt secret values at rest with a key from an environment variable, mark those fields write-only in the API so they are never returned, and exclude them from logs and error messages. **Do this before you put a real password in, not after.**

Step 6 — flake tolerance (2 hours)

Retry the whole transaction once before recording a failure. Multi-step flows fail occasionally for reasons that are not outages, and without this your incident list will be noise.

Step 7 — the step builder UI (3 hours)

A form that adds, reorders and removes steps. Keep it plain — a list of cards with inputs is entirely sufficient. Resist building a drag-and-drop editor; it is a day of work that adds nothing an examiner will credit.

What you learn

Interpreter and template-rendering patterns, JSONPath, encryption at rest, designing a user-facing configuration format, and the practical difference between testing a component and testing a journey.

EXIT TEST

- ☐ A three-step flow against a real application (use one of your own sites) passes end to end
- ☐ A token extracted in step 1 is correctly used in step 2's `Authorization` header
- ☐ Breaking step 2 deliberately produces a failure that names **step 2** and the assertion that failed
- ☐ The stored password is unreadable in the database and is absent from every API response and log line
- ☐ Total transaction duration is recorded and plotted over time

PHASE 9

Deploy correlation and the public status page

Week 11 · ~8 hours · Produces: two features with very high demo value

> **Goal:** deployments appear on the incident timeline, and anyone can view a public status page without logging in.

Deploy correlation (5 hours)

1. `V9__deploys.sql` : the `deploy_events` table, plus a `webhook_secret` on `project` .
2. `POST /webhook/deploy/{projectToken}` accepting commit, environment and actor.
3. HMAC-SHA256 signature verification over the **raw request bytes**, compared with `MessageDigest.isEqual` (Document 1 §7.5). Both details matter and both are common mistakes.
4. Store your own receipt time as authoritative; keep any client-supplied time in a separate column.
5. The correlation query: for an incident, find deploys in the 30 minutes before it opened.
6. Surface it on the incident detail page, worded as correlation: “a deploy occurred 4 minutes before this incident.”
7. **Bonus, ~30 minutes:** the change failure rate query (Document 1 §5.7). One SQL statement over data you already have, and it is a DORA metric. Put it on the dashboard.

Public status page (3 hours)

1. A `public` boolean on `checks` and a `slug` on `project` .
2. `GET /status/{slug}` — unauthenticated, returning only public checks.
3. Cache the response for 30–60 seconds. Explain why in a comment: peak traffic coincides exactly with the outage.
4. A simple React page: overall banner, per-component state, a 90-day history strip, current incidents.

What you learn

HMAC verification and constant-time comparison, raw request body handling in Spring, caching strategy driven by traffic shape, and building a genuinely public surface on a multi-tenant system.

EXIT TEST

- ☐ A curl POST with a correct signature is accepted; the same POST with a wrong signature returns 401
- ☐ A deploy sent just before you break a check appears on that incident’s timeline
- ☐ The status page loads in a logged-out private browser window and shows only checks marked public
- ☐ An internal-only check is confirmed absent from the public page

PHASE 10

Retention, hardening, documentation, demo

Week 12 · ~10 hours · Produces: a submission, not just a codebase

> **Goal:** the system is operationally complete, the report is written, and the demo is rehearsed. **Write no new features this week.**

Tasks

1. **Retention job** (2 h). The nightly rollup and delete from Document 1 §8.2, deleting in batches. Run it once manually and record the before-and-after row counts — that comparison goes straight into your report.
2. **Self-monitoring** (30 min). Point a heartbeat check and an HTTP check at your own platform's `/actuator/health`. It demonstrates you understood your own product, and it is a pleasing thing to show.
3. **Correlated-failure suppression** (1 h, optional but high value). If an unusually large proportion of all checks fail simultaneously, suspect yourself and suppress alerts. Answers viva question Q32 with working code rather than a plan.
4. **Error-path review** (1 h). Break things on purpose: wrong database password, unreachable email provider, malformed webhook body. Nothing should return a stack trace to a caller, and nothing should kill a scheduler thread silently.
5. **Collect the evidence** (1 h). Screenshot the longest uptime graph you have, a real incident with its full timeline, an alert email, and the deploy-correlated incident. These artefacts are worth more than any live demo.
6. **Write the report** (3 h). §4.4 has the structure. Your `decisions.md` is the design chapter.
7. **Rehearse the demo twice, out loud, against a timer** (1 h). §4.5 is the script.
8. **README** (30 min). Someone must be able to clone and run it. Include the `docker compose up` command and the environment variables required.

> **THE RULE FOR WEEK 12**
>
> No new features. A half-finished feature added in the final week is a liability in a viva: it creates a question you cannot answer well and a bug you have not found yet. Finish, document, rehearse.

EXIT TEST — YOU ARE DONE WHEN

- ☐ The retention job has run and you can state the row-count reduction as a number
- ☐ The platform has been running unattended for at least three weeks and you have the graph to prove it
- ☐ You have a real incident in the database with a deploy correlated to it
- ☐ You can deliver the eight-minute demo without notes
- ☐ A classmate can clone the repository and run it from the README alone

3. Reference

The API surface, the full schema, dependencies and the tests that matter. Build from these rather than inventing as you go.

3.1 Complete API surface

Everything under `/api/v1` requires a JWT. Everything else is deliberately public.

Authentication

Endpoint	Phase	Purpose
`POST /api/v1/auth/register`	5	Create an account; returns tokens
`POST /api/v1/auth/login`	5	Exchange credentials for an access and refresh token
`POST /api/v1/auth/refresh`	5	New access token from a refresh token

Projects and checks

Endpoint	Phase	Purpose
`GET /api/v1/projects`	5	List projects owned by the caller
`POST /api/v1/projects`	5	Create a project (generates slug and webhook secret)
`GET /api/v1/checks`	1	List checks; filter by `projectId`, `type`, `state`
`POST /api/v1/checks`	1	Create a check — validates config against the type, runs the SSRF guard
`GET /api/v1/checks/{id}`	1	One check with its current state and last run
`PATCH /api/v1/checks/{id}`	1	Update name, interval, thresholds, config
`DELETE /api/v1/checks/{id}`	1	Delete the check and cascade its runs
`POST /api/v1/checks/{id}/run`	1	Run now, out of schedule
`POST /api/v1/checks/{id}/pause`	3	Move to `PAUSED`; stops scheduling
`POST /api/v1/checks/{id}/rotate-token`	6	New heartbeat token, keeping history

Results, incidents and alerts

Endpoint	Phase	Purpose
`GET /api/v1/checks/{id}/runs`	1	Paged run history; `from`, `to` filters

Endpoint	Phase	Purpose
`GET /api/v1/checks/{id}/stats`	3	Uptime %, avg and p95 latency over a window
`GET /api/v1/incidents`	3	List; filter `status=open|closed`
`GET /api/v1/incidents/{id}`	3	Detail: timeline, failing runs, correlated deploys
`POST /api/v1/incidents/{id}/ack`	4	Acknowledge; stops reminder escalation
`GET /api/v1/alert-rules`	4	List notification rules for a project
`POST /api/v1/alert-rules`	4	Add a channel and target
`GET /api/v1/deploys`	9	Recent deploy events for a project
`GET /api/v1/projects/{id}/change-failure-rate`	9	DORA metric over a window

Public endpoints — no authentication by design

Endpoint	Phase	Purpose
`POST /ping/{token}`	6	Heartbeat success. Returns 204. Idempotent via `X-Ping-Id`.
`POST /ping/{token}/start`	6	Job started — enables duration measurement
`POST /ping/{token}/fail`	6	Job actively reporting its own failure
`POST /webhook/deploy/{projectToken}`	9	Deploy event; requires HMAC signature header
`GET /status/{slug}`	9	Public status page data; cached 30–60 s
`GET /actuator/health`	0	Liveness; what your platform monitors about itself

3.2 Complete database schema

Split these across Flyway migrations in the phase order shown, never as one file.

```
-- V1: users and projects  (Phase 0)
CREATE TABLE app_user (
    id             BIGSERIAL PRIMARY KEY,
    email          TEXT NOT NULL UNIQUE,
    password_hash  TEXT NOT NULL,
    display_name   TEXT,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE project (
    id             BIGSERIAL PRIMARY KEY,
    user_id        BIGINT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
    name           TEXT NOT NULL,
    slug           TEXT NOT NULL UNIQUE,      -- public status page address
    webhook_secret TEXT NOT NULL,             -- HMAC key for deploy webhooks
    created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```
-- V2 + V3 + V4: the central table  (Phases 1-3, 6)
CREATE TABLE checks (
    id                     BIGSERIAL PRIMARY KEY,
    project_id             BIGINT NOT NULL REFERENCES project(id) ON DELETE CASCADE,
    type                   TEXT NOT NULL,      -- HTTP|TLS|DOMAIN|TCP|HEARTBEAT|SYNTHETIC
    name                   TEXT NOT NULL,
    enabled                BOOLEAN NOT NULL DEFAULT TRUE,
    is_public              BOOLEAN NOT NULL DEFAULT FALSE,

    interval_seconds       INT  NOT NULL,
    grace_period_seconds   INT  NOT NULL DEFAULT 0,
    failure_threshold      INT  NOT NULL DEFAULT 3,
    success_threshold      INT  NOT NULL DEFAULT 2,

    state                  TEXT NOT NULL DEFAULT 'PENDING',
    consecutive_failures   INT  NOT NULL DEFAULT 0,
    consecutive_successes  INT  NOT NULL DEFAULT 0,

    next_run_at            TIMESTAMPTZ,        -- pull checks: when to probe next
    next_expected_at       TIMESTAMPTZ,        -- heartbeat: deadline to receive a ping
    token                  TEXT UNIQUE,        -- heartbeat only

    config                 JSONB NOT NULL DEFAULT '{}',
    created_at             TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Partial indexes: the scheduler's two questions, answered in milliseconds.
CREATE INDEX idx_checks_due
    ON checks (next_run_at) WHERE enabled AND next_run_at IS NOT NULL;
CREATE INDEX idx_checks_overdue_heartbeat
    ON checks (next_expected_at) WHERE enabled AND type = 'HEARTBEAT';
```

```
-- V2: results  (Phase 1)
CREATE TABLE check_runs (
    id              BIGSERIAL PRIMARY KEY,
    check_id        BIGINT NOT NULL REFERENCES checks(id) ON DELETE CASCADE,
    started_at      TIMESTAMPTZ NOT NULL,
    success         BOOLEAN NOT NULL,
    latency_ms      INT,
    status_code     INT,
    failure_kind    TEXT,        -- TIMEOUT|DNS_FAILURE|CONNECTION_REFUSED|...
    detail          TEXT,        -- short human-readable; TRUNCATE, never full bodies
    days_remaining  INT,         -- TLS and DOMAIN checks only
    failed_step     INT,         -- SYNTHETIC checks only
    dedupe_key      TEXT         -- heartbeat idempotency
);

CREATE INDEX idx_runs_check_time ON check_runs (check_id, started_at DESC);
CREATE UNIQUE INDEX uq_runs_dedupe
    ON check_runs (check_id, dedupe_key) WHERE dedupe_key IS NOT NULL;
```

```
-- V4: incidents  (Phase 3)
CREATE TABLE incidents (
    id               BIGSERIAL PRIMARY KEY,
    check_id         BIGINT NOT NULL REFERENCES checks(id) ON DELETE CASCADE,
    opened_at        TIMESTAMPTZ NOT NULL,
    closed_at        TIMESTAMPTZ,
    cause            TEXT,
    acknowledged_at  TIMESTAMPTZ,
    acknowledged_by  BIGINT REFERENCES app_user(id)
);

-- At most one OPEN incident per check — enforced by the database,
-- so a bug in the state machine cannot produce duplicates.
CREATE UNIQUE INDEX uq_one_open_incident
    ON incidents (check_id) WHERE closed_at IS NULL;
```

```
-- V5: alerting  (Phase 4)
CREATE TABLE alert_rules (
    id          BIGSERIAL PRIMARY KEY,
    project_id  BIGINT NOT NULL REFERENCES project(id) ON DELETE CASCADE,
    channel     TEXT NOT NULL,      -- EMAIL | WEBHOOK
    target      TEXT NOT NULL,      -- address or URL
    enabled     BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE alert_deliveries (
    id              BIGSERIAL PRIMARY KEY,
    incident_id     BIGINT NOT NULL REFERENCES incidents(id) ON DELETE CASCADE,
    alert_rule_id   BIGINT REFERENCES alert_rules(id) ON DELETE SET NULL,
    channel         TEXT NOT NULL,
    transition_type TEXT NOT NULL,   -- OPENED | REMINDER_n | RECOVERED
    sent_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    status          TEXT NOT NULL,   -- SENT | FAILED
    detail          TEXT
);

-- The at-most-once guarantee.
CREATE UNIQUE INDEX uq_alert_once
    ON alert_deliveries (incident_id, channel, transition_type);
```

```
-- V9: deploy events  (Phase 9)
CREATE TABLE deploy_events (
    id           BIGSERIAL PRIMARY KEY,
    project_id   BIGINT NOT NULL REFERENCES project(id) ON DELETE CASCADE,
    received_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  -- YOUR clock: authoritative
    reported_at  TIMESTAMPTZ,                         -- their clock: informational
    commit_sha   TEXT,
    environment  TEXT,
    actor        TEXT,
    pipeline_url TEXT
);
CREATE INDEX idx_deploy_project_time ON deploy_events (project_id, received_at DESC);

-- V10: rollups  (Phase 10)
CREATE TABLE check_run_hourly (
    check_id       BIGINT NOT NULL REFERENCES checks(id) ON DELETE CASCADE,
    bucket_start   TIMESTAMPTZ NOT NULL,
    total_runs     INT NOT NULL,
    failed_runs    INT NOT NULL,
    avg_latency_ms INT,
    p95_latency_ms INT,
    PRIMARY KEY (check_id, bucket_start)
);
```

3.3 Dependencies

```
<!-- Phase 0: the baseline -->
spring-boot-starter-web
spring-boot-starter-data-jpa
spring-boot-starter-validation
spring-boot-starter-actuator
org.postgresql:postgresql            (runtime)
org.flywaydb:flyway-core
org.flywaydb:flyway-database-postgresql   <-- separate artifact since Flyway 10.
                                              Forgetting it is a confusing startup failure. -->

<!-- Phase 5: security -->
spring-boot-starter-security
spring-boot-starter-oauth2-resource-server   <-- OR io.jsonwebtoken:jjwt-*; pick one -->

<!-- Phase 8: synthetic transactions -->
com.jayway.jsonpath:json-path        <-- JSONPath assertions and extraction -->

<!-- Testing, from Phase 1 onward -->
spring-boot-starter-test
org.testcontainers:postgresql
org.testcontainers:junit-jupiter
com.squareup.okhttp3:mockwebserver   <-- a fake HTTP server for probe tests:
                                         lets you simulate timeouts, 500s and slow
                                         responses deterministically -->
```

3.4 Testing strategy

You do not need high coverage. You need roughly a dozen tests that prove the parts that are hard to get right — and each one of them is an answer to a viva question.

#	Test	Phase	Proves
1	Two failures then a success opens no incident	3	Hysteresis works
2	Three consecutive failures open exactly one incident	3	Threshold is correct
3	Ten further failures still yield one incident	3	No duplicate incidents
4	Two successes close it with a `closed_at`	3	Recovery path works
5	User B gets 404 for user A's check	5	No IDOR — tenant isolation
6	Targets on localhost, 169.254.x and 10.x are rejected	5	SSRF guard works
7	The same `X-Ping-Id` twice creates one run row	6	Idempotency

#	Test	Phase	Proves
8	A heartbeat 1 s past its deadline is detected; 1 s before is not	6	Deadline arithmetic, via a fixed `Clock`
9	A probe against a server that never responds fails at the timeout, not later	2	Timeouts are enforced
10	A webhook with a tampered body is rejected	9	HMAC verification
11	A synthetic flow fails at step 2 and the result names step 2	8	Step-level diagnostics
12	Uptime over a window with one 30-minute incident computes correctly	3	The uptime formula

> **TWO TECHNIQUES THAT MAKE THESE POSSIBLE**
>
> **Inject a `Clock`.** Never call `Instant.now()` inside logic you need to test. Declare a `Clock` bean, use `Instant.now(clock)`, and in tests supply `Clock.fixed(...)`. Tests 8 and 12 are impossible without this and trivial with it.
>
> **Use a mock HTTP server.** MockWebServer lets you script exact responses — a 500, a 3-second delay, a malformed body — so probe tests are deterministic and do not depend on the internet being up or on somebody else's site behaving.

3.5 Frontend screens

Eight screens is the whole application. Build them in this order, each immediately after the API it consumes exists.

Screen	After phase	Contents
Login / Register	5	Two forms; stores the token; redirects
Dashboard	3	Grid of check cards: name, type, state colour, uptime %, last-run sparkline. Polls every 30 s.
Check detail	3	Latency line chart, 90-day uptime strip, recent runs table, incident history
Create / edit check	1	Type selector that swaps the config fields. The one screen worth spending design effort on.
Synthetic step builder	8	Ordered list of step cards with add, remove and reorder
Incidents	3	Open and resolved, with duration and cause
Incident detail	9	Timeline with failing runs, alerts sent, and correlated deploy markers. **This is your demo screenshot.**
Public status page	9	No navigation, no login, one clear banner plus per-component state

Keep the visual design plain and consistent: one accent colour, green / amber / red for state, generous whitespace. A clean, restrained dashboard reads as more professional than an elaborate one, and every hour spent on animation is an hour not spent on the engine that the viva is actually about.

4. Delivery

The calendar, what to cut under pressure, the risks, the report and the demo.

4.1 Week-by-week calendar

Dates assume a start in the week of 22 September 2026. Adjust the dates, keep the order.

Week	Dates	Phase	Visible outcome at the end of the week
1	22–28 Sep	0 — Skeleton and deploy	A public URL that answers `/api/v1/ping`
2	29 Sep–5 Oct	1 — One check end to end	Create a check by API, run it manually, see the stored result
3	6–12 Oct	2 — The engine	Checks run by themselves; history accumulates overnight
4	13–19 Oct	3 — Incidents	Break a site, watch one incident open and close
5	20–26 Oct	4 — Alerting	A real notification arrives on your phone
6	27 Oct–2 Nov	5 — Auth, tenancy, SSRF	Two accounts; neither can see the other's data
7	3–9 Nov	6 — Heartbeat	A real cron job on your machine is monitored
8	10–16 Nov	7 — TLS, domain, TCP	Six check types; engine untouched
9	17–23 Nov	8a — Synthetic (execution)	A multi-step flow runs and passes
10	24–30 Nov	8b — Synthetic (variables, UI)	Token flows between steps; builder UI works
11	1–7 Dec	9 — Deploy + status page	A deploy marker sits beside an incident on the timeline
12	8–14 Dec	10 — Polish and documents	Report written, demo rehearsed, nothing new added

> **BUILD IN A BUFFER NOW, NOT LATER**
>
> Assume you will lose about a week to exams, illness or a bug that eats three evenings — because you will. The plan already absorbs this: Phase 7 can compress to three days and Phase 9 to one. Do not respond to slipping by cutting Phase 10; an undocumented, unrehearsed project demos badly however good the code is.

4.2 If you only have eight weeks — the cut list

Cut in this order. Each line is a decision you can defend in a viva as a scoping choice rather than a failure, **provided you say so in your report**.

Cut	What goes	Time saved	Cost to the project
1st	Domain expiry check	1 day	Almost none — structurally near-identical to the TLS check
2nd	TCP reachability check	0.5 day	Almost none — the simplest probe of the six

Cut	What goes	Time saved	Cost to the project
3rd	Public status page	3 hours	Small; loses one nice demo screen
4th	Email channel (keep webhook only)	4 hours	Small — the strategy pattern still demonstrates extensibility with one channel plus a stub
5th	Retention and rollup job	2 hours	Moderate — but you can still present the arithmetic and the designed policy, which answers the question

> NEVER CUT THESE
>
> **Phase 2 (the scheduler)** — without it the project is a CRUD app with a fetch button. **Phase 3 (incidents and hysteresis)** — the intellectual core. **Phase 5's SSRF guard** — two hours of work that produces your single best viva answer. **Phase 8 (synthetic transactions)** — the feature that distinguishes this from an off-the-shelf uptime monitor.

4.3 Risk register

Risk	Likelihood	Early warning sign	Mitigation
Free host sleeps, scheduler stops	High	Gaps in run history overnight	Check the host's idle policy in week 1. Budget for a paid instance for the final month, or use an always-on VM.
Free database deleted after 30 days	High	An expiry email you did not read	Note the creation date in week 1. Take a `pg_dump` weekly; it is one command and it also protects your demo evidence.
Outbound SMTP blocked	Medium	Email works locally, silently fails deployed	Use an email provider's HTTP API from the start, never raw SMTP.
Synthetic transactions overrun	Medium	Week 10 arrives and variables still do not substitute	Ship it in the layered order in Phase 8. Execution without variables is already demonstrable.
Spring Boot 4 documentation gaps	Medium	Tutorials that do not compile	Read the official reference rather than blogs. Decide 4.1 vs 3.5 in week 1 and never revisit it.
Frontend eats the backend's time	High	Two days spent on chart styling	API first, always. Timebox each screen to one evening; plain is fine.
Nothing to demo because nothing broke	Medium	An incident list that is empty	Deliberately break something in week 6 and preserve that incident. Keep a check pointed at a URL you control so you can break it on demand.

4.4 What goes in the report

A structure that maps onto standard software-engineering documentation, with a note on where the material comes from.

Chapter	Length	Source
1. Introduction and problem statement	3–4 pp	Document 1 Part 2 — the three failure stories, then the unifying insight
2. Literature / existing systems	3–4 pp	Document 1 §2.3 — compare honestly with Healthchecks.io, UptimeRobot, PagerDuty; state your differentiator
3. Requirements (SRS)	5–6 pp	Functional requirements from the phase exit tests, which are already written as verifiable statements. Non-functional: 60 s detection latency, 1-minute minimum interval, tenant isolation.
4. System design	8–10 pp	Document 1 Parts 3, 4 and 6, with the figures. Include the ER diagram, the architecture diagram and the state machine.
5. Implementation	8–10 pp	Per feature: why needed, how it connects, how it fetches. Document 1 Part 5 is organised exactly for this.
6. Design decisions	3–4 pp	Your `decisions.md`. **The chapter examiners read most closely.** One JSON column vs seven tables; one scheduler vs many timers; push vs pull; black-box vs agent.
7. Security	3–4 pp	Document 1 Part 7. SSRF with the code and the test. This chapter alone will distinguish your report.
8. Testing	2–3 pp	§3.4's twelve tests, with what each proves and the results
9. Results and evaluation	3–4 pp	Real numbers: weeks of uptime collected, incidents detected, mean detection time, rows before and after compaction, lines changed when adding a check type
10. Limitations and future work	2–3 pp	Single-region, DNS rebinding, TCP reachability ≠ health, no agent-based metrics. **Naming real limitations raises your mark rather than lowering it.**

> THE ONE CHAPTER THAT LIFTS A REPORT FROM GOOD TO EXCELLENT
>
> Chapter 9. Almost every student report describes what was built; very few measure it. You will have real numbers — days of continuous operation, actual incidents detected with timestamps, storage before and after compaction, the count of files changed when adding the sixth check type. Put those numbers in a table. **Evidence beats assertion in every viva.**

4.5 The eight-minute demo script

Rehearse this twice against a timer. Have everything open in tabs beforehand — never log in or navigate menus while an examiner watches.

Time	What you show	What you say
0:00	The dashboard, already loaded, with real data	“This has been running continuously for N weeks. Everything here was collected by the platform itself, unattended.”
0:45	The check list showing six different types	“Six check types, one engine. Adding the last three required three classes and no change to the scheduler, incidents or alerting.”
1:30	A check detail page with a multi-week uptime graph	“Uptime is computed from incident durations, not from counting runs — that is what an SLA actually measures.”
2:30	**Break something live.** Stop a service or point a check at a dead URL, then trigger ‘run now’ twice.	“Two failures and nothing has fired — that is hysteresis. One blip is not an outage.”
3:15	The third failure. The incident appears. Show the alert arriving.	“Third consecutive failure: one incident, one alert. Not three.”
4:00	The incident detail with a correlated deploy marker	“A deploy was recorded four minutes before this. It does not prove causation, but it is where I would look first.”
5:00	The synthetic transaction — steps, then a failure naming step 2	“Every individual endpoint returns 200. The journey is broken at step 2. That gap is why this feature exists.”
6:00	A terminal: the one-line curl in a real crontab	“Nothing is installed and no SSH is used. Five check types need only a URL; heartbeat needs one line.”
6:45	Try to create a check against `http://169.254.169.254` — rejected	“My core feature is fetching user-supplied URLs, which is SSRF by definition. This is the guard, and here is the test that proves it.”
7:30	Fix the broken check; the incident closes and a recovery notice arrives	“Recovery needs two consecutive successes, so a single lucky response does not close an incident.”

THREE RULES FOR THE DEMO

Lead with real accumulated data, not a fresh empty install. Weeks of autonomous history is your strongest single claim and it takes ten seconds to establish.

Break something live. Nothing else demonstrates that the system genuinely works. Rehearse the exact break so it is reliable, and have a screenshot ready as a fallback if the network misbehaves.

End on the SSRF guard. It is the last thing they hear, it is the most sophisticated thing in the project, and almost no other submission will have one.

A closing note

The order in this plan matters more than any individual detail in it. Most student projects fail not because the code was too hard but because the sequencing was wrong — six features built to 70% each, integrated in the final week, demoed as a collection of half-working screens.

The alternative is what this plan does: build one narrow path all the way through, make it genuinely work, and then let everything else become cheap. By the end of week 5 you will have a system that runs on its own, detects real failures and tells a human about them. Everything after that is addition rather than construction — and that is a very different kind of December.